

9. Архитектура ПО ИС

<https://yandex.ru/search/?text=Архитектура+программного+обеспечения+информационных+систем+-+что+это+такое%3F&lr=43&clid=2270455&win=547>

Архитектура программного обеспечения информационных систем — **это структура системы, описывающая её основные компоненты, взаимодействия между ними и правила их функционирования.** 2

Она помогает определить, как различные компоненты взаимодействуют друг с другом, какие технологии и инструменты использовать, а также как обеспечить безопасность и производительность. 1

Некоторые ключевые принципы архитектуры ПО:

- **Модульность.** Каждый компонент системы должен быть независимым и легко заменяемым. 13
- **Инкапсуляция.** Логика и данные внутри компонента должны быть скрыты от других компонентов, что повышает безопасность и упрощает тестирование. 13
- **Разделение ответственности.** Каждый компонент или сервис системы должен выполнять только одну функцию, что облегчает поддержку и развитие. 13
- **Абстракция.** Взаимодействие между модулями должно быть чётко определено и стандартизировано, что облегчает поддержку системы в дальнейшем. 3
- **Масштабируемость и производительность.** Архитектура должна предусматривать возможность масштабирования системы, а также эффективно использовать ресурсы. 3

Некоторые типы архитектур ПО:

- **Монолитная архитектура.** Все функциональные части системы (клиентская и серверная логика, базы данных и т. д.) собраны в одном приложении. Такой подход прост в реализации и тестировании на ранних стадиях разработки, но со временем становится сложно масштабировать и поддерживать систему. 13
- **Микросервисная архитектура.** Система разделяется на несколько независимых сервисов, каждый из которых отвечает за определённую функциональность. Каждый микросервис имеет свою собственную базу данных и управляется отдельно, что упрощает обновления и расширения. 13
- **Сервис-ориентированная архитектура (SOA).** Строится на основе взаимодействующих сервисов, которые могут быть реализованы на разных языках программирования или работать на разных платформах. Такой подход помогает повысить гибкость и масштабируемость системы. 13
- **Бессерверная архитектура.** Предполагает использование облачных функций или сервисов, которые позволяют выполнять код без необходимости управлять серверами, что упрощает развёртывание и поддержку. 1
- **Клиент-серверная архитектура.** Приложение делится на клиентскую и серверную части. 2

- **Событийно-ориентированная архитектура.** Компоненты взаимодействуют через события. [2](#)
- **Слоистая архитектура.** Программа организована в виде слоёв, например, *презентационный слой, бизнес-логика и база данных*. [2](#)

Некоторые преимущества монолитной архитектуры:

- **Простота разработки.** Все компоненты находятся в одном репозитории, что упрощает работу команды. Разработчики могут сосредоточиться на функциональности, а не на взаимодействии между сервисами. [13](#)
- **Единое развёртывание.** Нет необходимости управлять множеством сервисов, а обновление системы требует лишь одной операции деплоя. [1](#)
- **Лёгкость тестирования.** Общее тестирование всех компонентов можно проводить в одной среде. [1](#)
- **Удобство в небольших командах.** Подходит для стартапов или проектов с ограниченными ресурсами, где важно быстро приступить к работе. [1](#)

Некоторые недостатки монолитной архитектуры:

- **Сложность масштабирования.** Масштабирование требует копирования всего приложения, даже если нужно повысить производительность только одной его части. [12](#)
- **Зависимость компонентов.** Любое изменение в одном модуле может привести к непредсказуемым последствиям в других частях системы. [1](#)
- **Долгосрочная поддержка.** Переход на новые технологии требует переработки всей системы. [1](#)
- **Риск единой точки отказа.** Ошибка в одном компоненте может привести к полной остановке всего приложения. [1](#)

Некоторые альтернативы монолитной архитектуре:

- **Микросервисная архитектура.** Приложение состоит из отдельных независимых модулей (микросервисов). У каждого из них своя логика, база данных, язык кода. Микросервисы могут разрабатываться, развёртываться и масштабироваться независимо друг от друга. Этот подход подходит для крупных и сложных проектов, где важна масштабируемость и гибкость. [143](#)
- **Бессерверная архитектура.** Это облачное решение, где всю заботу о масштабировании и распределении ресурсов берёт на себя провайдер. Клиенту остаётся только работа с функциями. Бессерверная архитектура идеальна для небольших и средних облачных приложений с непредсказуемыми нагрузками. [2](#)
- **Гибридная архитектура.** Это компромисс между микросервисами и монолитом. Основная часть функционала приложения реализована в виде монолита, но для некоторых частей используются микросервисы. Например, сайт может быть реализован в виде монолита, а мобильное приложение — отдельным микросервисом. [1](#)

Выбор альтернативы монолитной архитектуре зависит от конкретных требований проекта. Разработчики должны тщательно взвешивать преимущества и недостатки

каждого подхода, учитывая требования проекта, его масштаб, командную структуру и будущие планы развития. 53

Некоторые преимущества микросервисной архитектуры:

- **Гибкое масштабирование.** Можно увеличивать мощности точечно, только там, где это действительно необходимо. Например, во время распродаж можно масштабировать только сервис обработки заказов, не затрагивая остальные компоненты системы. 34
- **Удобные обновления.** Если нужно что-то обновить, можно изменить только несколько сервисов и не трогать остальные. Не понадобится переписывать всё, ведь модули независимы. 3
- **Отказоустойчивость.** Микросервисы независимы друг от друга. Поэтому если один или даже несколько модулей выйдут из строя, это не повлияет на работу остальных. 3
- **Возможность выбирать технологии.** С микросервисами продукт не «привязан» к конкретному языку программирования и стеку инструментов. 3
- **Независимость данных.** Данные каждого микросервиса хранятся отдельно от других, поэтому изменения в модели данных одного модуля не затрагивают остальные. 3
- **Наглядный контроль качества.** Задачи легче распределить между командами. Каждый делает что-то своё, его результаты можно наглядно оценить. 3
- **Упрощённый процесс отладки и обслуживания.** Возможные проблемы обычно касаются лишь одного сервиса, что позволяет оперативно локализовать и исправить проблему без влияния на остальные компоненты системы. 1

Для управления микросервисами используют различные инструменты, среди них:

- **Инструменты оркестрации.** Помогают организовывать и управлять взаимодействием между микросервисами. Разработчики определяют взаимодействие служб, настраивают протоколы связи и обеспечивают эффективную обработку данных. 12
- **Решения для обнаружения сервисов.** Например, Consul или Eureka. Инструмент обеспечивает связь между службами путём сопоставления зависимых служб и управления их отношениями. 12
- **Очереди сообщений.** Система обмена сообщениями, которая используется микросервисами для надёжного и асинхронного взаимодействия друг с другом. 12
- **API-шлюзы.** Платформа для управления внешним доступом к микросервисной архитектуре. Обеспечивает безопасную маршрутизацию запросов, аутентификацию, авторизацию, ограничение скорости, кэширование, мониторинг и т. д.. 12
- **Инструменты непрерывной интеграции и доставки.** Позволяют разработчикам быстро создавать и развёртывать приложения в различных средах с минимальными усилиями. Поддерживают процедуры автоматического

тестирования, которые позволяют проверять новый код перед выпуском в производственную среду. [1](#)

- **Инструменты мониторинга.** Помогают отслеживать доступность, производительность и масштабируемость приложений в различных конфигурациях среды. [1](#)

Некоторые популярные инструменты для управления микросервисами:

- **Kubernetes.** Платформа с открытым исходным кодом, которая позволяет легко запускать контейнерные приложения в любой облачной инфраструктуре. [13](#)
- **Docker.** Позволяет создавать переносимые образы компонентов приложения, которые затем можно использовать на нескольких компьютерах. [13](#)
- **Istio и Envoy.** Инструменты Service Mesh с открытым исходным кодом, которые обеспечивают безопасную и надёжную связь между микросервисами. [1](#)
- **Prometheus.** Система мониторинга микросервисов с открытым исходным кодом, которая помогает разработчикам быстро устранять проблемы с производительностью. [1](#)

Выбор подходящего API-шлюза для микросервисной архитектуры зависит от множества факторов, включая требования к производительности, существующий стек технологий, опыт команды. [1](#)

Некоторые рекомендации по выбору:

- **Учитывать поддержку необходимых функций.** Нужно оценить, соответствует ли один шлюз потребностям или требуется ли несколько. [3](#)
 - **Рассмотреть встроенные предложения.** Их можно использовать, когда они соответствуют требованиям к безопасности и контролю. Настраиваемый шлюз стоит выбирать, если встроенные параметры не имеют необходимой гибкости. [3](#)
 - **Выбрать нужную модель развёртывания.** Например, управляемые службы, такие как шлюз приложений Azure и управление API Azure, могут снизить рабочие накладные расходы. [3](#)
 - **Учитывать управление изменениями.** При обновлении служб или добавлении новых служб может потребоваться обновить правила маршрутизации шлюза. [3](#)
- Несколько популярных API-шлюзов:

- **Amazon API Gateway.** Облачное решение от AWS, интегрируется с Lambda, DynamoDB и другими сервисами. [2](#)
- **Apigee.** Платформа от Google с мощными возможностями мониторинга, безопасности и аналитики. [2](#)
- **Kong.** Высокопроизводительный API-шлюз с поддержкой плагинов и интеграцией с Kubernetes. [25](#)
- **Spring Cloud Gateway.** Решение на Java, интегрированное с Spring Boot, поддерживает реактивную обработку запросов. [2](#)
- **WSO2 API Microgateway.** Открытый облачный, разработчикский и децентрализованный API-шлюз для микросервисов. [5](#)

Каждое из перечисленных решений имеет свои сильные стороны и ограничения, поэтому выбор должен основываться на конкретных потребностях проекта. [1](#)

Архитектура информационной системы состоит из трёх компонентов 14:

1. **Слой представления 14.** Клиентская часть с графическим интерфейсом пользователя, которая выполняет роль терминала — средства представления данных и отправки команд 1.
2. **Слой бизнес-логики 14.** Здесь происходит обработка команд, полученных от клиента, и выполняются основные вычисления 1. Чаще всего это реализуется в виде серверного приложения 1. Здесь же располагается система управления базой данных (СУБД) как надстройка над базой данных, которая позволяет обратиться к данным и манипулировать ими 1.
3. **Слой доступа к данным 14.** Сама база данных как хранилище данных в структурированном виде, что в конечном итоге на низком уровне сводится к файлам с записанными данными 1

Различают четыре вида программно-аппаратной архитектуры информационных систем (ИС) 1:

1. **Централизованная 1.** Предполагает использование одного выделенного компьютера (с несколькими терминалами) для хранения и управления данными, а также для выполнения прикладных функций информационной системы 1.
2. **Файл-серверная 13.** Хранение данных осуществляется на выделенной машине, а все остальные операции — на клиентских станциях, подключённых по локальной сети 1.
3. **Клиент-серверная 13.** Хранение и управление данными осуществляется на отдельном компьютере — сервере БД 1. Клиентские рабочие станции отвечают за логику приложений ИС и интерфейсы пользователей 1.
4. **Интегрированная 1.** Не предполагает чёткого разделения компонентов системы на клиентские части и серверные 1. В такой архитектуре любой компонент может быть как сервером, так и клиентом 1.

Некоторые уровни, которые входят в архитектуру информационных систем:

1. **Информационно-логический уровень 3.** Представляет собой совокупность потоков данных и центров (узлов) возникновения, потребления и модификации информации 3.
2. **Прикладной уровень 3.** Совокупность прикладных программ и программных комплексов, которые реализуют функционирование информационно-логической модели 3.
3. **Системный уровень 3.** Операционные системы и сетевые средства 3.
4. **Аппаратный уровень 3.** Средства вычислительной техники 3.
5. **Транспортный уровень 3.** Активное и пассивное сетевое оборудование, сетевые протоколы и технологии 3.

Также компоненты информационной системы по выполняемым функциям можно разделить на **три слоя**: слой представления, слой бизнес-логики и слой доступа к данным [45](#).

Некоторые компоненты, которые входят в архитектуру базы данных:

- **Словарь или каталог данных** [1](#). Служит для централизованного накопления и описания ресурса данных [1](#). Содержит описание предметной области, сведения о структуре базы данных и связях между её элементами [1](#).
- **Система управления базами данных (СУБД)** [1](#). Совокупность языковых и программных средств, предназначенных для создания, ведения и совместного использования базы данных многими пользователями [1](#).
- **Ядро СУБД** [4](#). Отвечает за управление данными во внешней памяти, буферами оперативной памяти, транзакциями и журнализацию [4](#).
- **Компилятор языка базы данных** [4](#). Компилирует операторы языка базы данных в выполняемую программу [4](#).
- **Подсистема поддержки времени исполнения** [4](#). Интерпретирует программы манипуляции данными, создающие пользовательский интерфейс с СУБД [4](#).
- **Сервисные программы (внешние утилиты)** [4](#). В них выделяют такие процедуры, которые слишком накладно выполнять с использованием языка базы данных, например, загрузка и выгрузка базы данных, сбор статистики, глобальная проверка целостности базы данных [4](#).

Виды архитектур информационной системы

Основные виды архитектур информационных систем (ИС) [15](#):

1. **Централизованная** [1](#). Предполагает использование одного выделенного компьютера (с несколькими терминалами) для хранения и управления данными, а также для выполнения прикладных функций информационной системы [1](#).
2. **Файл-серверная** [15](#). Хранение данных осуществляется на выделенной машине, а все остальные операции — на клиентских станциях, подключённых по локальной сети [1](#).
3. **Клиент-серверная** [15](#). Хранение и управление данными осуществляется на отдельном компьютере — сервере БД [1](#). Клиентские рабочие станции отвечают за логику приложений ИС и интерфейсы пользователей [1](#).
4. **Интегрированная** [1](#). Не предполагает чёткого разделения компонентов системы на клиентские части и серверные [1](#). В такой архитектуре любой компонент может быть как сервером, так и клиентом [1](#).